

Documentation — Phase 1: WHAT

Purpose. Define the solution architecture for **Health Informatics — Hospital Readmission Risk**. No code yet. The mentor reviews and signs off this document before implementation begins.

Team

Role	Name
Team lead	Bekzod Adkhmjanov
Member 2	Ruslan Mamatanov
Member 3	Ibrohim Tohirjonov
Member 4	Nilufar Nazarova

Date: **Chosen scenario:** Health Informatics — Hospital Readmission Risk

1. Problem Restatement

In **your own words**, restate the business problem in 3–5 sentences.

The hospital network wants to explore reasons for diabetes patients returning within 30 days after treatment. The goal is to examine anonymized admissions across different hospitals to find a relationship between features and readmissions. Cleaning data and preparing data will be the most important step

2. Proposed Solution Stack

List the **specific tools, libraries, and methods** you plan to use at each stage.

- Python, pandas, numpy, matplotlib, seaborn

2.1 Data download & ingestion

- *direct CSV download, store under data/raw*

2.2 Data cleaning/hygiene

- *Standardize Missing Values — `df.replace()`*
- *Drop Sparse or Irrelevant Columns — `df.drop()`*

- *Linearize Age Ranges* — `df.map()`
- *Binarize Target Variable* — `df.apply()`
- *Group ICD-9 Codes* — Custom Function + `.apply()`
- *Deduplicate Patients* — `df.drop_duplicates()`

2.3 Analysis / modelling

- *Categorical Grouping*
- *Feature Ranking*
- *Binary Classification*
- *Feature Correlation*
- *Patient Segmentation*

2.4 Visualization

- *Horizontal Bar Chart* — Rank feature importance.
- *Stacked Bar Chart* — Compare readmission proportions.
- *Heatmap* — Analyze feature correlations.
- *Box Plot* — Detect numerical outliers.
- *Grouped Bar Chart* — Compare ICD-9 groups.
- *Histogram* — View age distributions.

2.5 Reporting

This project is all about figuring out why some diabetes patients end up back in the hospital within 30 days of leaving. We're looking at a huge pile of data from 130 different hospitals to see if there are common patterns, like specific treatments or patient habits, that lead to these quick returns. The biggest part of the job is going to be "cleaning" the data—basically, fixing messy entries and organizing everything so we can actually make sense of it.

Our plan is to use Python and a few handy libraries to get the data ready and build our analysis. We'll start by tossing out useless info (like columns with way too many blanks) and grouping complicated medical codes into simple categories like "Heart" or "Digestion". Once the data is tidy, we'll rank the factors to see which ones are the biggest red flags for a patient coming back. Finally, we'll turn those findings into easy-to-read charts—like bar graphs and heatmaps—so the hospital can see exactly where the risks are.

3. Logic Map

Describe the data flow in stages. Each stage = **input** → **transformation** → **output**.

Stage 1 – Ingestion

Input: Raw CSW file

Action: Load via Pandas; initial schema inspection

Output: Raw DataFrame

Stage 2 – Cleaning

Input: Raw DataFrame

Action: Apply hygiene rules(drop **weight**, fix **?**, map **age**, snake_case),Group ICD-9 code

Output: Cleaned “Hygiene-Ready” DataFrame

Stage 3 – Analysis

Input: Cleaned DataFrame

Action: Run feature importance ranking

Output: List of top 5 predictive features

Stage 4 – Visualization

Input: Feature importance results

Action: Generate important charts showing the relationship between features and readmission

Output: Set of 4-5 key plots

Stage 5 – Insight & Recommendation

Input: Visualizations and rankings

Action: Connect data patterns to actual hospital care changes

Output: Insights into why patients return and how to possibly prevent it

Replace each ... with concrete details for **Health Informatics — Hospital Readmission Risk**.

4. Workload Split (3 Developers)

Developer	Responsibility	Stages owned
1 Bekzod	Clean, Insights	2, 5
2 Ibrohim	Features	3
3 Ruslan	Visualize	4
4 Nilufar	Uploading code	1

5. Risks & Mitigation

List 3 things that could go wrong and your mitigation plan.

Risk: Deleting too much data during the "Drop Sparse Columns" step could remove variables that actually influence readmission.

Mitigation: Set a strict threshold (like 40% or 50% missingness) before dropping a column to ensure only truly empty data is removed.

Risk: "Deduplicate Patients" might accidentally remove a patient's legitimate return visit if they were readmitted more than once.

Mitigation: Keep only the first chronologically documented encounter for each unique patient ID to ensure a fair "first-readmission" analysis.

Risk: "Group ICD-9 Codes" could oversimplify medical conditions, causing different diseases to be lumped into the same category.

Mitigation: Use standard clinical grouping ranges (like the 390–459 rang

6. Sign-off

- ☐ All 3 team members reviewed and agree
- ☐ Mentor reviewed and approved
- Mentor signature / date: